

Rule of Three-Five-Zero

1. The core problem: resource ownership

Suppose a class manually manages a resource:

```
class Buffer {
private:
    int* data;

public:
    Buffer(int value) {
        data = new int(value);
    }

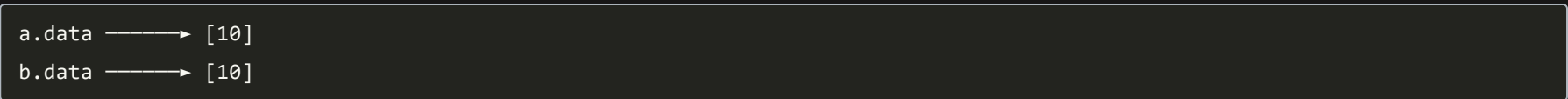
    ~Buffer() {
        delete data;
    }
};
```

Here, `Buffer` owns dynamically allocated memory.

Now consider:

```
Buffer a(10);
Buffer b = a;
```

The compiler-generated copy constructor does a **shallow copy**:



Both objects point to the **same memory**.

When they are destroyed:

```
a destructor → delete data
b destructor → delete data again ✗
```

That's **double deletion / undefined behavior**.

This is why special member functions matter.

2. Rule of Three

The **Rule of Three** says:

If a class needs to explicitly define **any one** of these three, it probably needs to explicitly define **all three**:

- 1. Destructor
- 2. Copy constructor
- 3. Copy assignment operator

Why?

Because if you're manually managing a resource, you need to correctly handle:

OperationFunctionObject destructionDestructorCreating object from anotherCopy constructorAssigning existing objectCopy assignment

Example:

```
class Buffer {
    int* data;

public:
    Buffer(int value) {
        data = new int(value);
    }

    // 1. Destructor
    ~Buffer() {
        delete data;
    }

    // 2. Copy constructor
    Buffer(const Buffer& other) {
        data = new int(*other.data);
    }

    // 3. Copy assignment
    Buffer& operator=(const Buffer& other) {
        if (this != &other) {
            delete data;
            data = new int(*other.data);
        }
        return *this;
    }
};
```

Now:

```
Buffer a(10);

Buffer b = a;    // copy constructor
Buffer c(20);

c = a;          // copy assignment
```

Each object owns its **own memory**.

```
a.data → [10]
b.data → [10]
c.data → [10]
```

3. Copy Constructor vs Copy Assignment

This is a **very common** interview question.

Copy constructor

Creates a **new object**:

```
Buffer b = a;
```

or

```
Buffer b(a);
```

Signature:

```
Buffer(const Buffer& other);
```

Copy assignment

Object **already exists**:

```
Buffer b(20);
b = a;
```

Signature:

```
Buffer& operator=(const Buffer& other);
```

Easy mental rule

Constructor = initialization
Assignment = replacing an existing object

4. Rule of Five

C++11 introduced **move semantics**.

Now there are five special member functions:

1. Destructor
2. Copy constructor
3. Copy assignment
4. Move constructor
5. Move assignment

So:

If a class manually manages a resource and needs one of the special functions, it may need all **five**.

The new two are:

```
Buffer(Buffer&& other);           // move constructor

Buffer& operator=(Buffer&& other); // move assignment
```

5. Why do we need move semantics?

Consider:

```
Buffer createBuffer() {
    return Buffer(100);
}

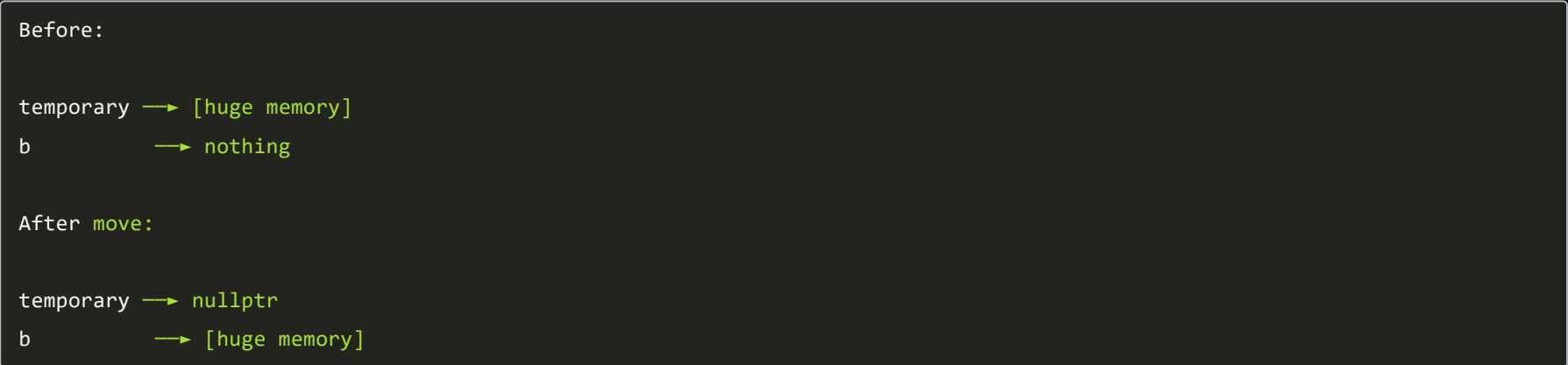
Buffer b = createBuffer();
```

Suppose **Buffer** owns a huge array:



A copy would require allocating another huge block and copying everything.

Move semantics can simply transfer ownership:



No expensive deep copy.

6. Move Constructor

Example:

```
Buffer(Buffer&& other) {
    data = other.data;
    other.data = nullptr;
}
```

Notice:

```
data = other.data;
```

We steal the pointer.

Then:

```
other.data = nullptr;
```

We make sure the old object no longer owns it.

This is called **transferring ownership**.

7. Move Assignment

For an already-existing object:

```
Buffer& operator=(Buffer&& other) {  
    if (this != &other) {  
        delete data;  
  
        data = other.data;  
        other.data = nullptr;  
    }  
  
    return *this;  
}
```

Usage:

```
Buffer a(10);  
Buffer b(20);  
  
b = std::move(a);
```

Afterward:

```
b → [10]  
a → nullptr
```

8. Complete Rule of Five Example

```

class Buffer {
    int* data;

public:
    Buffer(int value) {
        data = new int(value);
    }

    // 1. Destructor
    ~Buffer() {
        delete data;
    }

    // 2. Copy constructor
    Buffer(const Buffer& other) {
        data = new int(*other.data);
    }

    // 3. Copy assignment
    Buffer& operator=(const Buffer& other) {
        if (this != &other) {
            delete data;
            data = new int(*other.data);
        }
        return *this;
    }

    // 4. Move constructor
    Buffer(Buffer&& other) noexcept {
        data = other.data;
        other.data = nullptr;
    }

    // 5. Move assignment
    Buffer& operator=(Buffer&& other) noexcept {
        if (this != &other) {
            delete data;

            data = other.data;
            other.data = nullptr;
        }

        return *this;
    }
};

```

Interview point

You'll often see:

```
noexcept
```

on move operations.

That's important because standard containers such as `std::vector` may prefer moving an object during reallocation only when the move constructor is `noexcept` (or when copying isn't available).

9. Rule of Zero ★

This is arguably the **most important modern C++ rule**.

If your class doesn't directly manage a resource, don't write these special member functions yourself.

Instead, use **RAII classes** such as:

```

std::string
std::vector
std::unique_ptr
std::shared_ptr

```

Example:

Bad / unnecessary manual resource management

```
class Person {
    char* name;

public:
    // manually write destructor
    // copy constructor
    // copy assignment
    // move constructor
    // move assignment
};
```

Better

```
class Person {
    std::string name;

public:
    Person(std::string n) : name(std::move(n)) {}
};
```

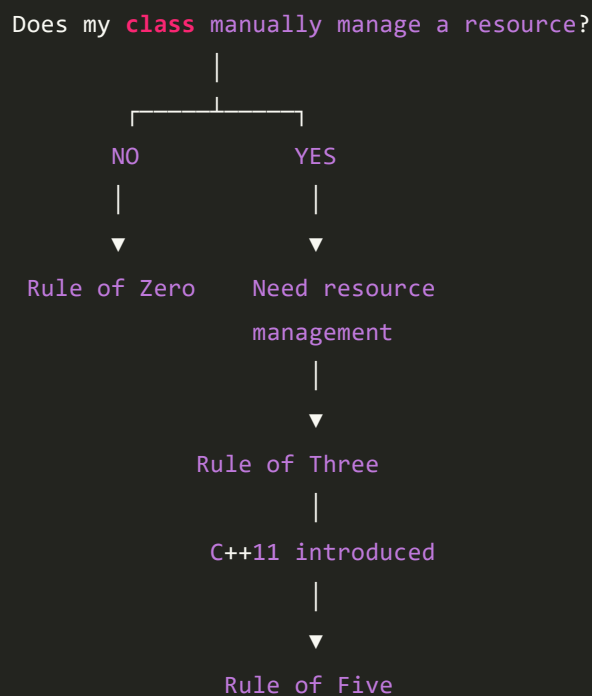
`std::string` manages its own memory.

You don't need to manually implement the five functions.

That's **Rule of Zero**.

10. The hierarchy you should remember

Think of it like this:



In modern C++:

Prefer Rule of Zero.

Only implement the Rule of Five when you genuinely need custom resource ownership/behavior.

11. What counts as a resource?

Not just heap memory.

Examples:

- Dynamic memory
- File handles
- Sockets
- Mutex locks
- Database connections
- OS handles
- GPU resources

General principle:

If your class owns something that must be explicitly released, think Rule of Three/Five.

12. Important interview distinction

Rule of Three

C++98-era idea:

```
Destructor
Copy constructor
Copy assignment
```

Rule of Five

C++11+:

```
Destructor
Copy constructor
Copy assignment
Move constructor
Move assignment
```

Rule of Zero

Modern best practice:

```
Don't manually manage resources.
Use RAII/resource-managing classes.
```



Interview Questions You Should Be Able to Answer

Q1. What is the Rule of Three?

Answer:

If a class requires a custom destructor, copy constructor, or copy assignment operator because it manages a resource, it generally needs all three to correctly implement ownership and copying.

Q2. Why does a class with a destructor often need a copy constructor?

Because a destructor usually indicates **manual resource ownership**.

If the compiler performs a shallow copy, two objects may own the same resource, causing issues such as **double deletion**.

Q3. What are the five special member functions?

```
Destructor
Copy constructor
Copy assignment operator
Move constructor
Move assignment operator
```

Q4. What's the difference between copy and move?

Copy:

Duplicate the **resource**

Move:

Transfer ownership of the **resource**

For a pointer:

```
// copy
data = new int(*other.data);

// move
data = other.data;
other.data = nullptr;
```

Q5. Why is `std::move` needed?

`std::move` doesn't actually move anything by itself.

It essentially casts an object to an rvalue reference so that a move constructor/assignment can be selected.

```
b = std::move(a);
```

means:

"I'm allowing you to treat `a` as something whose resources can be transferred."

Q6. What is Rule of Zero?

A class should avoid manually managing resources and instead delegate resource management to RAII types like `std::string`, `std::vector`, and smart pointers, allowing the compiler-generated special member functions to work correctly.

Q7. Why prefer Rule of Zero?

Because manually implementing the Rule of Five is:

- error-prone
- verbose
- difficult to maintain
- easy to get wrong with exceptions/self-assignment/ownership

RAII types handle these details for you.

🌀 One interview trap

If interviewer asks:

"If I define a destructor, will the compiler still generate the move constructor?"

This is where things get subtle.

A user-declared destructor can prevent implicit move operations from being generated.

That's one reason blindly writing a destructor can interfere with move semantics.

So in modern C++:

Don't write a destructor just because you can. Prefer Rule of Zero.

Final cheat sheet

RuleFunctionsRule of 3Destructo + Copy ctor + Copy assignmentRule of 5Rule of 3 + Move ctor + Move assignmentRule of 0Don't manually manage resources

And the most important interview mental model:

```
Resource ownership
  ↓
Need custom cleanup?
  ↓
YES → Think Rule of 5
NO  → Prefer Rule of 0
```

Interview priority: ★★★★★

Especially know shallow vs deep copy → Rule of Three → move semantics → Rule of Five → Rule of Zero.